

Session du 23 Avril 2026 — Maison Intelligente GEMMA

Maison Intelligente — Cahier de session

Projet : Système GEMMA — Architecture AppDaemon / Home Assistant

Participant : Joanny & Claude (partenaire technique)

1. Vision du projet

Une maison intelligente ne reçoit pas d'ordres. Elle **observe, comprend** et **déduit** l'état de l'humain pour adapter son environnement et atteindre des objectifs de confort.

Principe directeur : *la maison ne gère pas des événements, elle atteint des objectifs de confort définis par le profil de vie de l'occupant.*

Point clé de la session : l'index.yaml est un **inventaire physique** (ce qui existe), pas un fichier de comportements. Les comportements sont dans un fichier séparé.

2. Architecture générale — les 4 étapes

```
personnes.yaml → déclare qui existe
person_repository.py → accès aux données personne
Presence.py → calcule qui est à la maison
↓
index.yaml → inventaire du matériel (capteurs + actionneurs)
↓
1. signal_reader.py → lecture des états en temps réel → snapshot.json
↓
2. moteur_gemma.py → calcul de l'état humain → gemma_state.json
poids.yaml → poids ajustables par signal
↓
3. moteur_scoring.py → décision et action sur les actionneurs
comportements.yaml → matrice état × météo
↓
Actionneurs HA → lumières, switches, futur : rideaux, chauffage
```

3. Fichiers produits cette session

Fichier	Rôle	Statut
index.yaml	Inventaire matériel (existant)	Existe ✓
signal_reader.py	Lecteur d'états temps réel + présence	Créé ✓
moteur_gemma.py	Calcul état GEMMA par scoring	Créé ✓
poids.yaml	Poids des signaux par état (ajustable)	Créé ✓
moteur_scoring.py	Lecture météo + actions sur actionneurs	Créé ✓
comportements.yaml	Matrice état × météo → valeurs cibles	Créé ✓
apps.yaml	Configuration AppDaemon	Créé ✓

Tous les fichiers AppDaemon se déposent dans `/config/appdaemon/apps/`

L' `index.yaml` reste à la racine : `/config/index.yaml`

4. Détail — `signal_reader.py`

Rôle

Lit l' `index.yaml` , s'abonne à toutes les entités Home Assistant, interroge la brique `Presence` pour la présence humaine, et produit un `snapshot.json` mis à jour à chaque changement d'état.

Le `snapshot.json` produit

```
{
  "generated_at": "2026-04-23T14:32:01",
  "signals": {
    "Tele": {
      "entity": "media_player.65pus7304_12_2",
      "piece": "Salon",
      "type": "media_player",
      "raw": "playing",
      "value": "playing",
```

```

        "updated_at": "2026-04-23T14:31:55"
    },
    "Capteur luminosité": {
        "value": 32.0
    },
    "maison_occupee": {
        "value": true
    },
    "presence_Joanny": {
        "value": true
    }
}

```

Intégration avec Presence.py

Signal Reader appelle `self.get_app("presence")` pour accéder à la brique Presence existante — sans dupliquer la logique de présence. Chaîne des briques :

```
personnes.yaml → person_repository → Presence → signal_reader
```

API publique exposée

```

get_snapshot()           → dict complet en mémoire
get_signal(signal_id)    → valeur normalisée d'un signal
get_lux()                → float lux courant (point d'entrée unique)

```

5. Détail — moteur_gemma.py + poids.yaml

Rôle

Lit le `snapshot.json` toutes les 30 secondes, calcule un score pour chaque état GEMMA, et publie l'état gagnant.

Formule

$$\text{Score} = \Sigma(\text{signal} \times \text{poids} \times \text{modificateur_contexte}) / \Sigma(\text{poids_max})$$

Si score > seuil (0.75) → état confirmé
Sinon → INDETERMINE

États GEMMA reconnus

État	Description
ABSENT	Personne à la maison
SOMMEIL	Humain en train de dormir
REPOS_TV	Humain devant la télévision
REPOS_MUSIQUE	Humain qui écoute de la musique
NUTRITIF	Humain en cuisine
ENTRETIEN	Humain en ménage ou réparation
ACTIF_BUREAU	Humain travaillant au bureau
INDETERMINE	Aucun état ne dépasse le seuil

Modificateurs contextuels (heure)

Plage	Modificateur
Nuit (23h → 7h)	×1.3 — amplifie les signaux de sommeil
Soirée (20h → 23h)	×1.1
Journée (7h → 20h)	×1.0

Publication du résultat

GEMMA publie l'état détecté à deux endroits :

- `gemma_state.json` — lu par le moteur de scoring
- `input_text.gemma_etat_courant` — visible dans le dashboard HA

À ajouter dans `configuration.yaml` :

```
input_text:
  gemma_etat_courant:
    name: GEMMA – État courant
    max: 50
```

6. Détail — moteur_scoring.py + comportements.yaml

Rôle

Lit `gemma_state.json` + météo HA + `sun.sun` , consulte `comportements.yaml` , et actionne les lumières et switches.

Mapping météo

Condition Météo France	Clé comportements
<code>sun.sun = below_horizon</code>	<code>nuit</code>
<code>sunny / clear + temp > 22°C</code>	<code>degage_chaud</code>
<code>sunny / clear + temp ≤ 22°C</code>	<code>degage_froid</code>
<code>cloudy / partlycloudy / fog</code>	<code>nuageux</code>
<code>rainy / pouring / lightning</code>	<code>pluvieux</code>

Exemple de flux complet

```
21h30 – TV s'allume (playing), temps nuageux
↓
signal_reader → snapshot : Tele = "playing"
moteur_gemma → score REPOS_TV = 0.94 → gemma_state.json
moteur_scoring → REPOS_TV × nuageux → comportements.yaml
→ Eclairage_Salon_tv : 60
→ Wled_Salon : 50
→ Spot_Ambiance : 30
→ turn_on() sur chaque entité HA
↓
La maison ajuste les lumières. Sans intervention manuelle.
```

7. Scalabilité — poids.yaml

Le fichier `poids.yaml` est la clé de l'évolution du système. Ajuster les comportements ne nécessite jamais de toucher au code Python — uniquement ce fichier.

Structure d'un signal dans `poids.yaml`

```
REPOS_TV:
  signaux:
    Tele:          { poids: 0.95, valeur_active: "playing" }
    Ampli:         { poids: 0.5,  valeur_active: true  }
    Capteur luminosité: { poids: 0.3,  seuil_max: 80 }
```

Trois types de critères d'évaluation :

- `valeur_active` — correspondance exacte (bool ou string)
- `seuil_max` — la valeur doit être $\leq$ seuil (ex: lux $\leq$ 80)
- `seuil_min` — la valeur doit être $\geq$ seuil (ex: lux $\geq$ 100)

8. Backlog — améliorations futures

Priorité	Élément	Note
1	Dashboard Lovelace GEMMA	Voir état courant, score, météo détectée
2	Aqara FP2	À commander — présence zone + activité (assis, allongé...)
3	Régulation par lux	<code>get_lux()</code> déjà en place dans <code>signal_reader</code>
4	Sous-états SOMMEIL	Endormissement / profond / besoin / réveil — nécessite FP2
5	Portabilité profil	US-22 — profil de confort exportable
6	Rideaux motorisés	Budget
7	Chauffage domotisé	Budget / infrastructure

Intégration FP2 Aqara (quand disponible)

Le FP2 détecte présence, zone et activité (assis, debout, allongé). Il suffira d'ajouter ses entités dans `index.yaml` et les nouveaux signaux dans `poids.yaml`. Les fichiers Python ne changent pas.

```
# Exemple futur dans poids.yaml
SOMMEIL:
  signaux:
    FP2_Chambre_activite: { poids: 0.95, valeur_active: "still" }
    FP2_Chambre_zone:     { poids: 0.8,  valeur_active: "lit" }
```

9. Briques existantes intégrées

Brique	Rôle	Lien avec GEMMA
<code>personnes.yaml</code>	Déclare les occupants	Source de vérité identité
<code>person_repository</code>	Accès aux données personne	Appelé par Presence
Presence	Qui est à la maison	Appelé par <code>signal_reader</code>
<code>eteindre_maison</code>	Actions au départ	Appelé par Presence

Principe architectural : chaque brique ne connaît que sa voisine directe. Si Presence évolue (ex: intégration FP2), `signal_reader` ne change pas — il appelle toujours `get_app("presence")`.

10. Fichiers de déploiement

<code>/config/index.yaml</code>	(existant – à la racine HA)
<code>/config/appdaemon/apps/signal_reader.py</code>	✓ créé
<code>/config/appdaemon/apps/moteur_gemma.py</code>	✓ créé
<code>/config/appdaemon/apps/moteur_scoring.py</code>	✓ créé
<code>/config/appdaemon/apps/poids.yaml</code>	✓ créé
<code>/config/appdaemon/apps/comportements.yaml</code>	✓ créé
<code>/config/appdaemon/apps/apps.yaml</code>	✓ créé

Entity_id météo à vérifier dans Outils de développement → États → chercher weather.

Document généré le 23 Avril 2026 — Projet Maison Intelligente GEMMA

Pour imprimer en PDF : Ctrl+P → Enregistrer en PDF